

Financial Risk & Fraud Analysis

SQL Work

This document records the SQL analysis steps completed for the Financial Risk & Fraud Analysis project.

1. Database Selection

Database used: financialriskanalysis

The database contains the financial-risk tables used for joins, aggregation, classification, and analysis.

2. SQL Analysis Workflow

- **USE:** Select the project database before running the analysis queries.
- **JOIN:** Combine related tables using matching keys so information from different tables can be analyzed together.
- **CASE WHEN:** Create categories or labels based on conditions.
- **GROUP BY:** Aggregate records by a category such as customer type or loan status.
- **ORDER BY:** Sort the final results to make the highest or lowest values easier to identify.
- **VIEW:** Save a useful query as a reusable database view.
- **Stored Procedure:** Create a reusable MySQL procedure that accepts a parameter and returns filtered results.

3. Query: Overdue Loans by Customer Type

Purpose: Count overdue loans for each customer type.

```
USE FinancialRiskAnalysis;
```

```
SELECT
    ct.TypeName,
    COUNT(*) AS OverdueLoans
FROM loans l
```

```
JOIN accounts a
  ON l.AccountID = a.AccountID
JOIN customers c
  ON a.CustomerID = c.CustomerID
JOIN customer_types ct
  ON c.CustomerTypeID = ct.CustomerTypeID
JOIN loan_statuses ls
  ON l.LoanStatusID = ls.LoanStatusID
WHERE ls.StatusName = 'Overdue'
GROUP BY ct.TypeName
ORDER BY OverdueLoans DESC;
Result shown in the SQL screen: Large Enterprise = 28, Individual = 22, Small Business = 18
overdue loans.
```

4. Query: Average Loan Amount by Loan Status

Purpose: Calculate the average principal amount for each loan status.

USE FinancialRiskAnalysis;

```
SELECT
  ls.StatusName,
  ROUND(AVG(l.PrincipalAmount), 2) AS AvgLoanAmount
FROM loans l
JOIN loan_statuses ls
  ON l.LoanStatusID = ls.LoanStatusID
GROUP BY ls.StatusName
ORDER BY AvgLoanAmount DESC;
```

Result shown in the SQL screen: Paid Off = 52,664.97; Active = 51,943.77; Overdue = 49,135.81.

5. CASE WHEN Analysis

CASE WHEN was used to classify records according to a condition. For example, loan amounts or balances can be grouped into High, Medium, and Low categories.

```
SELECT
  AccountID,
  Balance,
```

```
CASE
  WHEN Balance >= 50000 THEN 'High'
  WHEN Balance >= 20000 THEN 'Medium'
  ELSE 'Low'
END AS BalanceCategory
FROM accounts;
```

6. GROUP BY Analysis

GROUP BY was used with aggregate functions such as AVG() and COUNT() to summarize the data by category.

```
SELECT
  AccountTypeID,
  ROUND(AVG(Balance), 2) AS AverageBalance
FROM accounts
GROUP BY AccountTypeID;
```

7. VIEW

A VIEW was created to save a reusable result that combines account and customer information.

```
CREATE VIEW CustomerAccounts AS
SELECT
  a.AccountID,
  a.CustomerID,
  a.Balance,
  c.FirstName,
  c.LastName
FROM accounts a
JOIN customers c
  ON a.CustomerID = c.CustomerID;
```

8. MySQL Stored Procedure

Because the project uses MySQL Workbench, the stored procedure uses MySQL syntax.

```
DELIMITER //
```

```

CREATE PROCEDURE GetCustomerAccounts(IN p_CustomerID INT)
BEGIN
    SELECT *
    FROM accounts
    WHERE CustomerID = p_CustomerID;
END //

DELIMITER ;

```

9. What Each SQL Step Does

SQL Concept	What It Does
USE	Selects the database.
JOIN	Connects related tables using matching keys.
WHERE	Filters records before aggregation.
CASE WHEN	Creates conditional categories.
COUNT()	Counts records.
AVG()	Calculates an average.
ROUND()	Rounds a numeric result.
GROUP BY	Groups rows for aggregation.
ORDER BY	Sorts the result.
VIEW	Stores a reusable query as a virtual table.
Stored Procedure	Stores reusable SQL logic that can accept parameters.

10. Final Project Workflow

Power Query → Data Cleaning

MySQL → SQL Analysis (JOIN, CASE WHEN, GROUP BY, VIEW, Stored Procedure)

Power BI → Data Modeling, DAX, and Dashboard

Screenshots from the SQL work are included as evidence of the completed analysis steps.

11. SQL Work Screenshots

SQL Screenshot 1

The screenshot shows the SQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'accounts' and 'customers'. The main query editor, titled 'Query 1', contains the following SQL code:

```
1 • Use FinancialRiskAnalysis;
2 • select c.CustomerID,
3     c.FirstName,
4     c.LastName,
5     a.AccountID,
6     a.Balance
7 from customers c
8 join accounts a
9 on c.CustomerID = a.CustomerID ;
```

Below the query editor, the 'Result Grid' displays the results of the query. The grid has columns for CustomerID, FirstName, LastName, AccountID, and Balance. The first two rows of data are visible:

CustomerID	FirstName	LastName	AccountID	Balance
11083	Elfreda	Sheppard	201595	21641.13
10181	Rosenda	Bartlett	201025	21398.11

SQL Screenshot 2

The screenshot shows the SQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'accounts' and 'customers'. The main query editor, titled 'Query 1', contains the following SQL code:

```
1 • Use FinancialRiskAnalysis;
2 • create view BalanceCategoryLevel as
3     select Balance ,
4     case
5     when Balance >= 50000 then 'High'
6     when Balance >= 20000 then 'Medium'
7     else 'Low'
8     end as BalanceCategory
9 from accounts ;
10
```

SQL Screenshot 3

Navigator: Schemas

Filter objects

financialriskanalysis

- Tables
 - account_statuses
 - account_types
 - accounts
 - addresses
 - branches
 - customer_types
 - customers
- Columns
 - CustomerID
 - FirstName
 - LastName
 - DateOfBirth
 - AddressID
 - CustomerType
- Indexes
- Foreign Keys
- Triggers
- loan_statuses
- loans
- transaction_types
- transactions

Query 1

```
1 • Use FinancialRiskAnalysis;
2 • select BalanceCategory , count(*)
3   from BalanceCategoryLevel
4  group by BalanceCategory;
```

Limit to 50000 rows

Result Grid

BalanceCategory	count(*)
Medium	497
High	819
Low	351

SQL Screenshot 4

Navigator: Schemas

Filter objects

financialriskanalysis

- Tables
 - account_statuses
 - account_types
 - accounts
- Columns
 - AccountID
 - CustomerID
 - AccountTypeID
 - AccountStatus
 - Balance
 - OpeningDate
- Indexes
- Foreign Keys
- Triggers
- addresses
- branches
- customer_types

Query 1

```
1 • Use FinancialRiskAnalysis;
2 • select AccountTypeID , avg(Balance) as avgBalance
3   from accounts
4  group by AccountTypeID
5 order by avgBalance desc;
```

Limit to 50000 rows

Result Grid

AccountTypeID	avgBalance
2	50205.18498507462
3	50204.20744408948
1	48522.61012698411
4	48382.036876712344
5	48162.52064896758

Result 49 x

Output

SQL Screenshot 5

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'transaction_types' and 'transactions'. The 'transaction_types' table is expanded, showing columns 'TransactionTypeID' and 'TypeName'. The 'Query 1' window displays the following SQL query:

```
1 • Use FinancialRiskAnalysis;  
2 • select  
3   tt.TypeName , count(*) from transactions t  
4   join transaction_types tt  
5   on t.TransactionTypeID = tt.TransactionTypeID  
6   group by tt.TypeName;
```

The 'Result Grid' shows the results of the query:

TypeName	count(*)
Withdrawal	20590
Deposit	21043
Payment	6950
Transfer	20656

SQL Screenshot 6

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'transaction_types' and 'transactions'. The 'transactions' table is expanded, showing columns 'TransactionTypeID', 'Amount', and 'SDAmount'. The 'Query 1' window displays the following SQL query:

```
1 • Use FinancialRiskAnalysis;  
2 • select round(avg(Amount),2) as AvgAmount ,  
3   round(stddev(Amount),2) As SDAmount  
4   from transactions;  
5
```

The 'Result Grid' shows the results of the query:

AvgAmount	SDAmount
2498.55	1443.54

SQL Screenshot 7

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays a tree view of the 'financialriskanalysis' database, including tables like 'loan_statuses' and 'loans'. The 'Query 1' window is open, displaying the following SQL query:

```
1 • Use FinancialRiskAnalysis;  
2 • select ls.StatusName , count(*) as LoanCount  
3   from loans l  
4   join loan_statuses ls  
5   on l.LoanStatusID = ls.LoanStatusID  
6   group by ls.Statusname  
7   order by LoanCount;
```

Below the query, the 'Result Grid' shows the results of the query:

StatusName	LoanCount
Overdue	68
Paid Off	114
Active	484

SQL Screenshot 8

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane displays a tree view of the 'financialriskanalysis' database, including tables like 'loan_statuses' and 'loans'. The 'Query 1' window is open, displaying the following SQL query:

```
1 • Use FinancialRiskAnalysis;  
2 • SELECT ROUND(SUM(CASE WHEN ls.StatusName = 'Overdue' THEN 1 ELSE 0 END)  
3   /COUNT(*) * 100, 2) AS OverdueRate  
4   FROM loans l  
5   join loan_statuses ls  
6   on l.LoanStatusID = ls.LoanStatusID;
```

Below the query, the 'Result Grid' shows the results of the query:

OverdueRate
10.21

SQL Screenshot 9

Navigator

SCHEMAS

Filter objects

co

financialriskanalysis

Tables

- account_statuses
- account_types
- accounts
- addresses
- branches
- customer_types
- customers
- loan_statuses
- Columns
 - LoanStatusID
 - StatusName
- Indexes
- Foreign Keys
- Triggers
- loans
- Columns
 - LoanID

Query 1

```

1 • Use FinancialRiskAnalysis;
2 • SELECT ROUND(SUM(CASE WHEN ls.StatusName = 'Overdue'
3   THEN 1.PrincipalAmount ELSE 0 END), 2) AS OverdueLoanAmount
4   FROM loans l
5   JOIN loan_statuses ls
6   ON l.LoanStatusID = ls.LoanStatusID;
7

```

Result Grid

OverdueLoanAmount
3341234.84

SQL Screenshot 10

Navigator

SCHEMAS

Filter objects

co

financialriskanalysis

Tables

- account_statuses
- account_types
- accounts
- addresses
- branches
- customer_types
- customers
- loan_statuses
- Columns
 - LoanStatusID
 - StatusName
- Indexes
- Foreign Keys
- Triggers
- loans
- Columns
 - LoanID
 - AccountID
 - LoanStatusID
 - PrincipalAmount
 - InterestRate

Query 1

```

1 • Use FinancialRiskAnalysis;
2 • SELECT at.TypeName, COUNT(*) AS OverdueLoans
3   FROM loans l
4   JOIN accounts a
5   ON l.AccountID = a.AccountID
6   JOIN account_types at
7   ON a.AccountTypeID = at.AccountTypeID
8   JOIN loan_statuses ls
9   ON l.LoanStatusID = ls.LoanStatusID
10  WHERE ls.StatusName = 'Overdue'
11  GROUP BY at.TypeName
12  ORDER BY OverdueLoans DESC;

```

Result Grid

TypeName	OverdueLoans
Savings	24
Business	16
Youth	14
Payroll	8
Checking	6

SQL Screenshot 11

Navigator: SCHEMAS

Filter objects

co

financialriskanalysis

Tables

account_statuses

account_types

accounts

addresses

branches

customer_types

Columns

CustomerTypeID

TypeName

Indexes

Foreign Keys

Triggers

customers

loan_statuses

Columns

LoanStatusID

StatusName

Indexes

Foreign Keys

Triggers

loans

Administration Schemas

Query 1

Limit to 50000 rows

```

1 • Use FinancialRiskAnalysis;
2 • SELECT ct.TypeName, COUNT(*) AS OverdueLoans
3 FROM loans l
4 JOIN accounts a
5 ON l.AccountID = a.AccountID
6 JOIN customers c
7 ON a.CustomerID = c.CustomerID
8 JOIN customer_types ct
9 ON c.CustomerTypeID = ct.CustomerTypeID
10 JOIN loan_statuses ls
11 ON l.LoanStatusID = ls.LoanStatusID
12 WHERE ls.StatusName = 'Overdue'
13 GROUP BY ct.TypeName
14 ORDER BY OverdueLoans DESC;
15

```

Result Grid

TypeName	OverdueLoans
Large Enterprise	28
Individual	22
Small Business	18

SQL Screenshot 12

Navigator: SCHEMAS

Filter objects

co

financialriskanalysis

Tables

account_statuses

account_types

accounts

addresses

branches

customer_types

Columns

CustomerTypeID

TypeName

Indexes

Foreign Keys

Triggers

customers

loan_statuses

Columns

LoanStatusID

StatusName

Administration Schemas

Query 1

Limit to 50000 rows

```

1 • Use FinancialRiskAnalysis;
2 • SELECT ls.StatusName, ROUND(AVG(l.PrincipalAmount), 2) AS AvgLoanAmount
3 FROM loans l
4 JOIN loan_statuses ls
5 ON l.LoanStatusID = ls.LoanStatusID
6 GROUP BY ls.StatusName
7 ORDER BY AvgLoanAmount DESC;

```

Result Grid

StatusName	AvgLoanAmount
Paid Off	52664.97
Active	51943.77
Overdue	49135.81